

Redbeat - Software Design Document

1. Introduction

1.1 Purpose

This Software Design Document (SDD) provides a detailed overview of the technical design and implementation approach for the software system.

The purpose of this document is to define the system architecture, technology choices, design decisions, development approach, and technical specifications required to transform the approved client requirements into a functional software solution.

This document serves as a reference for the development team throughout the software development lifecycle, ensuring that the system is designed, implemented, tested, and maintained according to the defined requirements.

1.2 Document Scope

This document covers the technical design and development considerations of the software system, including the structure, components, technologies, and implementation strategies used during development.

The scope of this document includes:

- System architecture and technical design.
- Frontend and backend technology decisions.
- Database design approach.
- Third-party service integrations.
- Application security considerations.
- Development tools and environments.
- Testing and deployment considerations.
- Maintenance and future expansion planning.

This document does not define business requirements or user expectations, as those are described separately in the client requirement documentation.

1.3 Intended Audience

This document is intended for individuals involved in the design, development, testing, deployment, and maintenance of the software system.

The intended audience includes:

- Software Developers

Responsible for implementing the frontend, backend, database, and system integrations based on the defined technical design.

- Software Designers and Architects

Responsible for reviewing system structure, architecture decisions, and technical approaches.

- Quality Assurance (QA) Team

Responsible for understanding system behavior and preparing appropriate testing strategies.

- Project Managers

Responsible for monitoring development progress and ensuring technical implementation aligns with project objectives.

- Future Maintenance Teams

Responsible for updating, improving, and supporting the software system after deployment.

1.4 System Overview

The software system is designed as a modern web-based application that provides users with a reliable, scalable, and user-friendly digital platform.

The system follows a structured architecture consisting of frontend, backend, database, and third-party service components.

The frontend layer is responsible for providing the user interface and handling user interactions. It communicates with backend services to process business operations and retrieve required data.

The backend layer manages application logic, authentication, data processing, and communication between different system components.

The database layer is responsible for storing and managing application data, including user information, system records, and business-related information.

The system may integrate with external services such as payment providers, analytics platforms, notification services, and cloud storage solutions to extend functionality and improve reliability.

The overall system design focuses on:

- Scalability to support future growth.
- Maintainability through structured development practices.
- Security through proper access control and data protection.
- Performance optimization for reliable user experiences.
- Flexibility for future feature expansion and system improvements.

2. Project Understanding

2.1 Client Requirements Strategy

The client requirements strategy defines the approach used to analyze, organize, and translate client expectations into technical solutions.

The development process begins by reviewing the client's business needs, functional requirements, design expectations, and operational goals. These requirements are then evaluated to determine appropriate system features, technical solutions, and implementation priorities.

The strategy focuses on:

- Understanding the client's primary objectives and expected outcomes.
- Identifying essential system features and user requirements.
- Translating business requirements into technical specifications.
- Prioritizing important functionalities based on business impact.
- Ensuring the final system solution aligns with client expectations.

The development team will maintain continuous communication with the client throughout the project lifecycle to validate requirements, gather feedback, and ensure the delivered system meets the intended purpose.

2.2 Business Goals Interpretation

The business goals interpretation defines how the development team understands the client's objectives and converts them into system capabilities.

The software solution is designed to support business growth by providing a reliable digital platform that improves user experience, operational efficiency, and future scalability.

The primary business goals identified include:

Improving User Experience

The system should provide users with a simple, intuitive, and efficient experience when interacting with the platform.

This includes:

- Clear navigation and user interface design.
- Fast and reliable system interactions.
- Accessible features across different devices.
- Smooth completion of user tasks.

Increasing Operational Efficiency

The system should reduce manual processes and improve management efficiency through digital solutions.

This includes:

- Automated business processes.
- Centralized data management.
- Improved information accessibility.
- Reduced operational complexity.

Supporting Business Growth

The system should be designed with future expansion in mind.

This includes:

- Supporting increasing numbers of users.
- Allowing additional features to be introduced.
- Providing flexible system architecture.
- Supporting future integrations.

Improving Data Management

The system should provide structured management of business and user-related information.

This includes:

- Reliable data storage.
- Accurate information processing.
- Secure handling of sensitive data.
- Improved access to important records.

2.3 System Objectives

The system objectives define the technical and functional goals that guide the development and implementation of the software solution.

The primary objectives of the system are:

1. Develop a Reliable Software Platform

The system shall provide a stable and dependable platform capable of supporting expected user activities and business operations.

The system should:

Operate consistently under normal usage conditions.

Minimize service interruptions.

Provide reliable access to system features.

2. Implement Scalable System Architecture

The system shall be designed with scalability in mind to support future growth and increased demand.

The system should:

Support additional users and data volume.

Allow future feature expansion.

Enable improvements without major system restructuring.

3. Provide Secure Data Management

The system shall protect user information and business data through appropriate security practices.

The system should:

Implement secure authentication mechanisms.

Restrict unauthorized access.

Protect sensitive information.

Maintain data integrity.

4. Deliver Maintainable Software Components

The system shall be developed using structured and maintainable approaches.

The system should:

Use organized code architecture.

Support future maintenance activities.

Allow developers to identify and resolve issues efficiently.

Maintain proper technical documentation.

5. Ensure Quality User Experience

The system shall provide an effective and user-friendly experience across supported platforms.

The system should:

Provide responsive interfaces.

Maintain consistent design standards.

Support accessibility requirements.

Optimize usability for different user groups.

6. Support Future System Integration

The system shall be designed to allow integration with external services and future technologies.

The system should:

Support third-party service connections.

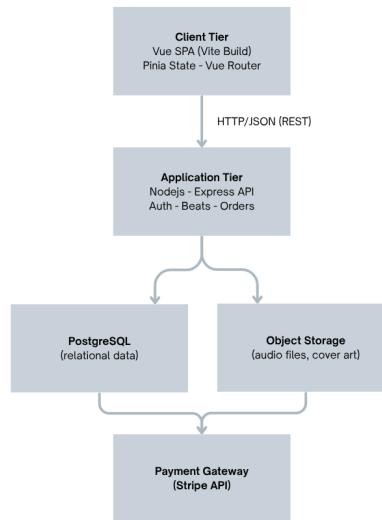
Allow additional features to be introduced.

Maintain flexibility for future technical improvements.

3. System Architecture Design

3.1 Overall System Architecture

Redbeat follows a standard three-tier architecture: a Vue.js single-page application on the client, a RESTful API layer handling business logic, and a persistence layer for structured data and media assets. The current prototype implements the client tier in full, with the API and database tiers represented by a local mock layer so the frontend can be developed and demoed independently; the sections below describe the target architecture the mock layer is designed to be swapped into.



Design rationale. Separating the SPA from the API server allows the two to scale and deploy independently — the frontend can be served from a CDN while the API scales horizontally behind a load balancer. Audio files are kept out of the relational database and stored as objects, since large binary blobs are expensive to query and back up inside a relational engine. Payment is delegated entirely to a third-party processor so Sidechain never handles raw card data directly, which also simplifies PCI compliance.

3.2 Frontend Architecture

3.2.1 User Interface Structure

Route	View	Purpose
/	Home	Landing page, hero, recently added beats
/register	Register	Artist account creation
/login	Login	Authentication
/marketplace	Marketplace	Browsable, filterable beat catalog
/beats/:id	Beat Detail	Single beat, preview player, purchase

Route	View	Purpose
/artists/:id	Dashboard	Authenticated artist's upload/ manage panel
/checkout	Checkout	Cart review and payment

The frontend is a single-page application with a persistent shell (top navigation bar and cart drawer) and a routed content area. Navigation is handled client-side via Vue Router with no full page reloads.

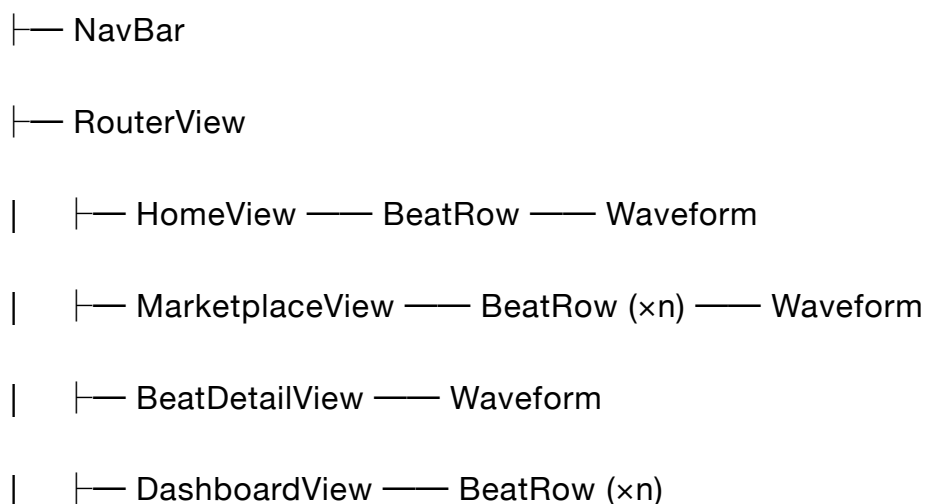
Route guards protect /dashboard, redirecting unauthenticated users to /login with a redirect query parameter so they return to their intended destination after signing in.

3.2.2 Component Architecture

Components follow a container/presentational split loosely modeled on atomic design:

Shell components (NavBar, CartDrawer) are mounted once in App.vue and persist across route changes. View components (HomeView, MarketplaceView, etc.) act as containers: they pull data from Pinia stores, handle route params, and compose presentational components. Presentational components (BeatRow, Waveform) are stateless and driven entirely by props, making them reusable across the marketplace, artist profile, and dashboard views without duplication.

App.vue



| └─ CheckoutView
└─ CartDrawer

All components use Vue 3's Composition API with `<script setup>`, which keeps component logic colocated and typed, and makes it straightforward to extract shared logic into composables as the codebase grows (for example, a future `useAudioPlayer()` composable could centralize playback state if multiple previews need to pause each other).

3.2.3 State Management Approach

Global state is managed with Pinia, split into three domain-scoped stores rather than one monolithic store:

auth store — current user session, login/register/logout actions
beats store — the beat catalog, including artist-uploaded listings
cart store — cart contents, totals, and the checkout action

This mirrors how the eventual API is organized by resource, so each store's internal implementation (currently backed by `localStorage`) can later be swapped for real HTTP calls without changing how components consume the store. Components never read or write persistence directly — they only call store actions and read store getters — which keeps the persistence strategy an implementation detail rather than something scattered across the UI.

3.3 Backend Architecture

3.3.1 Application Architecture

The target backend is a Node.js service built on Express, organized in layers to keep request handling, business rules, and data access separate:

Routes → define endpoints, attach middleware (auth, validation)

Controllers → parse requests, call services, shape responses

Services → business logic (e.g. "can this user list a beat?")

Repositories → data access, isolate all database queries

This layering means a controller never talks to the database directly, and a service never knows whether data comes from PostgreSQL or a cache — it only calls a repository interface. Middleware handles cross-cutting concerns: a JWT-verification middleware attaches the authenticated user to each request, a validation middleware rejects malformed payloads before they reach a controller, and a centralized error-handling middleware converts thrown exceptions into consistent JSON error responses.

File uploads (beat audio, cover art) are handled by an upload middleware that streams the file to object storage and stores only the resulting URL in the database, keeping large binaries out of the request/response cycle for every other endpoint.

3.3.2 API Architecture

Method	Endpoint	Description
POST	/api/v1/auth/register	Create an artist account
POST	/api/v1/auth/login	Authenticate, issue tokens
GET	/api/v1/beats	List/search/filter beats
GET	/api/v1/beats/:id	Get a single beat
POST	/api/v1/beats	Create a beat listing (auth required)
DELETE	/api/v1/beats/:id	Remove a listing (owner only)
GET	/api/v1/artists/:id	Public artist profile + catalog
POST	/api/v1/cart/checkout	Submit an order for payment
GET	/api/v1/orders	List the current user's past orders

The API is a versioned REST interface (/api/v1/...) using JSON for both requests and responses, with authentication via short-lived JWT bearer tokens issued at login and refreshed via a refresh-token endpoint.

Endpoints follow REST conventions (nouns for resources, HTTP verbs for actions), return standard HTTP status codes, and paginate list endpoints via ?page=/?limit= query parameters. Rate limiting is applied per IP and per authenticated user to protect the upload and checkout endpoints from abuse.

3.4 Database Architecture

3.4.1 Database Design Overview

The target persistence layer uses PostgreSQL for structured, relational data, chosen over a document store because Sidechain's core entities (users, beats, orders) have well-defined relationships and benefit from transactional guarantees during checkout — an order and its line items should either both be written or neither should be.

3.4.2 Data Relationships

A user can own many beats (one-to-many: beats.artist_id → users.id). A user can place many orders as a buyer (one-to-many: orders.buyer_id → users.id). An order contains many beats through order_items, and a beat can appear in many orders over time — a many-to-many relationship resolved by the order_items join table, which also snapshots the price at the time of purchase so later price changes don't rewrite history.

users —1—∞— beats

|

1

|

∞

orders —1—∞— order_items —∞—1— beats

3.4.3 Data Storage Strategy

Data is split across three storage types by access pattern:

Relational (PostgreSQL) — users, beats metadata, orders: structured, query-heavy, transactional data. Object storage (e.g. S3-compatible bucket) — audio files and cover art: large binary assets served directly via CDN URLs rather than through the API. Cache (e.g. Redis, optional) — session tokens and hot marketplace queries (such as "recently added beats") to reduce repeated database load on the homepage.

The current frontend-only prototype substitutes all of the above with the browser's localStorage, which is sufficient for demoing the full user flow (register → upload → browse → cart → checkout) without a live backend, but is explicitly not durable, multi-user, or secure, and is scoped to be replaced wholesale by the architecture above in a later milestone.

4. Technology Stack Specification

The technology stack defines the collection of programming languages, frameworks, databases, development tools, and external services used to design, develop, test, and maintain the software system.

The selected technologies are chosen based on factors such as scalability, maintainability, performance, developer productivity, community support, and future expansion requirements.

4.1 Frontend Technologies

HTML

HTML (HyperText Markup Language) is the standard markup language used to structure the content and layout of web applications. It is used to define the fundamental structure of the application, including pages, forms, navigation elements, and content sections.

CSS

CSS (Cascading Style Sheets) is used to control the visual presentation and styling of the application interface. It is responsible for managing layouts, colors, typography, animations, responsiveness, and overall user interface appearance.

TypeScript

TypeScript is a strongly typed programming language built on top of JavaScript that improves code reliability and maintainability. It is used for frontend application logic, providing better error detection and improved development experience.

TypeScript is selected because:

- It reduces common programming errors through static type checking.
- It improves code readability and maintainability.
- It provides better support for large-scale application development.
- It integrates well with modern frontend frameworks.

Vue

Vue.js is a progressive frontend JavaScript framework used to build interactive and component-based user interfaces.

Vue is responsible for managing dynamic website behavior, reusable components, and frontend application structure.

Vue is selected because:

- It provides a lightweight and efficient frontend development approach.
- It supports reusable component-based architecture.
- It has a clear learning curve and strong developer community.
- It provides flexibility for both small and large-scale applications.

Pinia

Pinia is a state management library designed specifically for Vue applications.

Pinia is used to manage shared application data such as user sessions, application states, and global information across different components.

Pinia is selected because:

- It provides a centralized approach for managing application states.
- It integrates naturally with Vue applications.
- It improves maintainability when applications become larger.
- It provides a simpler and more modern alternative to traditional state management approaches.

4.2 Backend Technologies

Python

Python is a high-level programming language used for backend development and application logic.

Python is used to handle server-side operations, business logic, data processing, and backend services.

Python is selected because:

- It provides rapid development capabilities.
- It has a large ecosystem of libraries and frameworks.
- It is suitable for web applications, automation, and data processing.
- It improves developer productivity through simple and readable syntax.

Golang

Golang (Go) is a programming language designed for building efficient, scalable, and high-performance backend services.

Golang is primarily used for microservices architecture and components that require high performance and scalability.

Golang is selected because:

- It provides excellent performance for server-side applications.
- It supports efficient concurrent processing.
- It is suitable for building independent microservices.
- It allows systems to scale effectively as user demand increases.

4.3 Database Technologies

Supabase

Supabase is an open-source backend platform that provides database management, authentication, APIs, and other backend services.

Supabase is used during the initial development phase to accelerate development and reduce infrastructure complexity

Supabase is selected because:

- It provides rapid backend setup.
- It includes built-in database and authentication features.
- It reduces initial development time.
- It is suitable for prototypes, MVPs, and early-stage applications.

MySQL

MySQL is a relational database management system used for storing and managing structured application data.

MySQL is planned as a future database solution when the system requires greater control, optimization, and enterprise-level scalability.

MySQL is selected because:

- It provides reliable relational data management.
- It supports complex data relationships.
- It is widely used in production environments.
- It provides strong performance for large-scale applications.

4.4 Third-Party Services

Third-party services are external platforms integrated into the system to provide additional functionality, improve performance, and support business operations.

Google Analytics

Google Analytics is used to collect and analyze website usage data.

It helps understand user behavior, traffic sources, and overall application performance.

Google Analytics is selected because:

- It provides detailed user activity insights.
- It helps measure website effectiveness.
- It supports data-driven decision making.
- It is widely adopted and well-supported

Google Ads

Google Ads is used for online advertising and user acquisition.

It allows the system to promote services or products through targeted advertising campaigns.

Google Ads is selected because:

- It provides access to a large advertising network.
- It supports targeted marketing strategies.
- It provides performance tracking for advertising campaigns.

Payment Gateway Services

Payment gateway services are used to enable secure online transactions between customers, the website, and financial service providers.

For Myanmar-based web applications, the system may integrate with local digital payment platforms and banking services to support convenient payment options for local users.

Potential payment integrations include:

KBZPay

KBZPay is one of Myanmar's widely used mobile wallet platforms.

It is integrated to support:

- Mobile wallet payments.
- QR-based payment transactions.
- Customer payment verification.
- Transaction status updates.

KBZPay integration is considered because of its large user base and popularity among Myanmar consumers.

Wave Money

Wave Money is a widely adopted mobile financial service in Myanmar.

It is used to support:

- Digital wallet payments.

- Customer transaction processing.
- Payment confirmation.
- Local customer payment preferences.

Wave Money integration improves accessibility for users who prefer mobile-based financial services.

Myanmar Banking Payment Services

The system may support integration with local banking services for customers who prefer direct bank transactions.

Supported integrations include:

- Bank transfer payments.
- QR payment systems.
- Online banking payment verification.

These integrations allow businesses to support customers who prefer traditional banking methods.

International Payment Gateways

International payment gateways may be integrated when global payment support is required.

Supported gateways include:

- Stripe.
- PayPal.
- Other international payment providers.

These services may support:

- International customers.
- Foreign currency transactions.
- Global payment methods.

Email and Notification Services

Email and notification services are used to provide automated communication between the system and users.

SendGrid

SendGrid may be used to provide reliable transactional email delivery.

Uses include:

- Account registration confirmation.
- Password reset emails.
- Order confirmation messages.
- Customer notifications.

Amazon Simple Email Service (Amazon SES)

Amazon SES may be used for scalable email communication.

It can support:

- High-volume email delivery.
- Automated system notifications.
- Business communication emails.

SMS and Mobile Notification Services

SMS notification services may be integrated for important user communications.

Uses include:

- Account verification codes.
- Login verification.
- Order status updates.
- Important system alerts.

For Myanmar users, SMS-based communication can be useful because mobile phone usage is widespread and provides an alternative communication channel when users do not frequently check email.

Push Notification Services

Push notification services may be used for mobile-friendly web applications or future mobile applications.

Uses include:

- Promotional announcements.
- Order updates.
- User activity alerts.
- Important reminders.

Cloud Storage Services

Cloud storage services are used to store and manage digital assets required by the application.

Instead of storing files directly on the application server, cloud storage provides better scalability, reliability, and performance.

Cloudflare R2

Cloudflare R2 is used as an alternative object storage solution.

It supports:

- Website media files.
- Image storage.
- Static assets.
- Cost-efficient file delivery.

Cloudflare R2 is useful for reducing storage costs while maintaining fast content delivery.

4.5 Development Tools and Environments

Visual Studio Code

Visual Studio Code (VS Code) is used as the primary integrated development environment for writing, testing, and managing source code.

VS Code is selected because:

- It supports multiple programming languages.
- It provides extensive extension support.
- It improves developer productivity.
- It integrates well with version control systems.

Git and Github

Git is used as the version control system for tracking source code changes.

GitHub is used as the remote repository platform for storing code, managing collaboration, and maintaining project history.

Git and GitHub are selected because:

- They provide reliable source code management.
- They support team collaboration.
- They allow tracking of development progress.
- They provide backup and version history.

Gitbook

GitBook is used for maintaining project documentation and technical references.

It is used to organize:

- Software documentation.
- Development guidelines.
- API documentation.
- System references.

GitBook is selected because:

- It provides structured documentation management.
- It supports collaboration between team members.
- It improves knowledge sharing and project maintainability.

Figma

Figma is used for designing user interfaces, creating wireframes, and preparing UI prototypes before development.

Figma is selected because:

- It supports collaborative design workflows.
- It allows rapid prototyping.
- It improves communication between designers and developers.

- It provides accurate visual references for implementation.

5. System Design Specification

5.1 User Management Design

User management covers artist/buyer registration, authentication, and profile data — in Redbeats there's a single users table rather than separate artist/buyer roles, since any registered user can both list beats and purchase them.

Registration flow: the client submits artist name, email, and password to POST /auth/register. The service layer checks the email isn't already taken, hashes the password with bcrypt, inserts the new user row, and returns a signed JWT pair so the user lands logged-in rather than being sent back to a login screen.

Login flow: POST /auth/login verifies the submitted password against the stored hash using bcrypt's constant-time comparison (to avoid timing attacks that could otherwise reveal whether an email exists), then issues a fresh token pair.

Session handling: the access token is attached automatically to every subsequent request via an HTTP client interceptor on the frontend; when a request comes back 401 Unauthorized, the interceptor silently attempts a refresh using the refresh token before falling back to logging the user out, so a short-lived access token doesn't interrupt an otherwise active session.

Profile editing: PATCH /users/me allows a logged-in user to update their artist name and bio; email changes are intentionally out of scope for the current design, since changing the account's login identifier safely requires a verification step not yet built.

5.2 Product Management Design

"Products" in Redbeats are beat listings. Each listing belongs to exactly one artist and carries the metadata a buyer needs to evaluate it before purchase: title, genre, BPM, key, price, and an audio preview URL.

Listing creation: an authenticated user submits listing details plus an audio file via POST /beats (multipart form data). The upload middleware streams the audio file to object storage first, then the controller inserts the beat row with the resulting URL — so a failed database write never leaves an orphaned, unreferenced audio file, and a failed upload never creates a beat row with a broken audio link.

Ownership rules: only the artist who created a listing can edit or delete it. This is enforced server-side (comparing the authenticated user's ID against beats.artist_id) rather than only hidden in the UI, so the rule holds even against a hand-crafted API request.

Catalog browsing: GET /beats supports filtering by genre and free- text search across title and artist name, plus pagination, so the marketplace view can request beats a page at a time rather than loading the entire catalog up front as it grows.

Soft vs. hard delete: deleting a listing is a hard delete in the current design (the row is removed). A future iteration could switch to a soft delete (an is_deleted flag) so beats that already appear in past orders remain viewable in a buyer's purchase history even after the artist removes them from the live catalog.

5.3 Shopping Cart Design

The cart holds beats a user intends to buy before checkout. Because digital beats are typically bought as singles rather than in quantity, a cart entry doesn't need a "quantity" field the way a physical-goods store would — each beat can only appear once per cart.

Guest vs. logged-in carts: for the current scope, only logged-in users have a cart, persisted server-side (cart_items, one row per user per beat) rather than only in browser storage, so a buyer's cart is available whether they return on the same device or a different one. Anonymous browsing is fully supported, but adding to cart prompts a login if the user isn't authenticated yet.

Adding/removing items: POST /cart/items and DELETE /cart/items/:beatId are idempotent — adding a beat already in the cart is a no-op rather than a duplicate

row, and removing a beat not in the cart simply returns success rather than erroring, which keeps the frontend free of special-casing around double-clicks or stale UI state.

Cart-to-checkout handoff: the cart is treated as a working list, not the order itself — checkout reads the current cart contents, but the order's line items are a separate, immutable snapshot (see 5.4), so changes to the cart after an order is placed can never retroactively alter a past order.

5.4 Order Management Design

An order represents a completed (or in-progress) purchase and exists independently of the cart from the moment checkout begins.

Order lifecycle: an order is created in a pending state as soon as checkout starts, moves to paid once the payment gateway confirms the charge, and moves to failed if payment is declined — so there's always a durable record of a checkout attempt, not just successful ones, which matters for support cases ("I was charged but never got the beat").

pending → paid → (fulfilled automatically — digital delivery)

pending → failed → (user may retry checkout)

Price snapshotting: each `order_items` row stores `price_at_purchase` independent of the beat's current listed price, so if an artist changes a beat's price after someone has bought it, historical orders and receipts remain accurate.

Digital fulfillment: because beats are digital, there's no shipping step — once an order reaches paid, the purchased beats' audio URLs become available to the buyer immediately via their order history and are served as full-quality downloads rather than the preview-quality stream used pre-purchase

5.5 Payment Integration Design6. Database Design Documentation

Payment processing is delegated entirely to a third-party gateway (designed around Stripe, though the pattern applies to comparable providers) rather than handling card data directly.

Checkout flow:

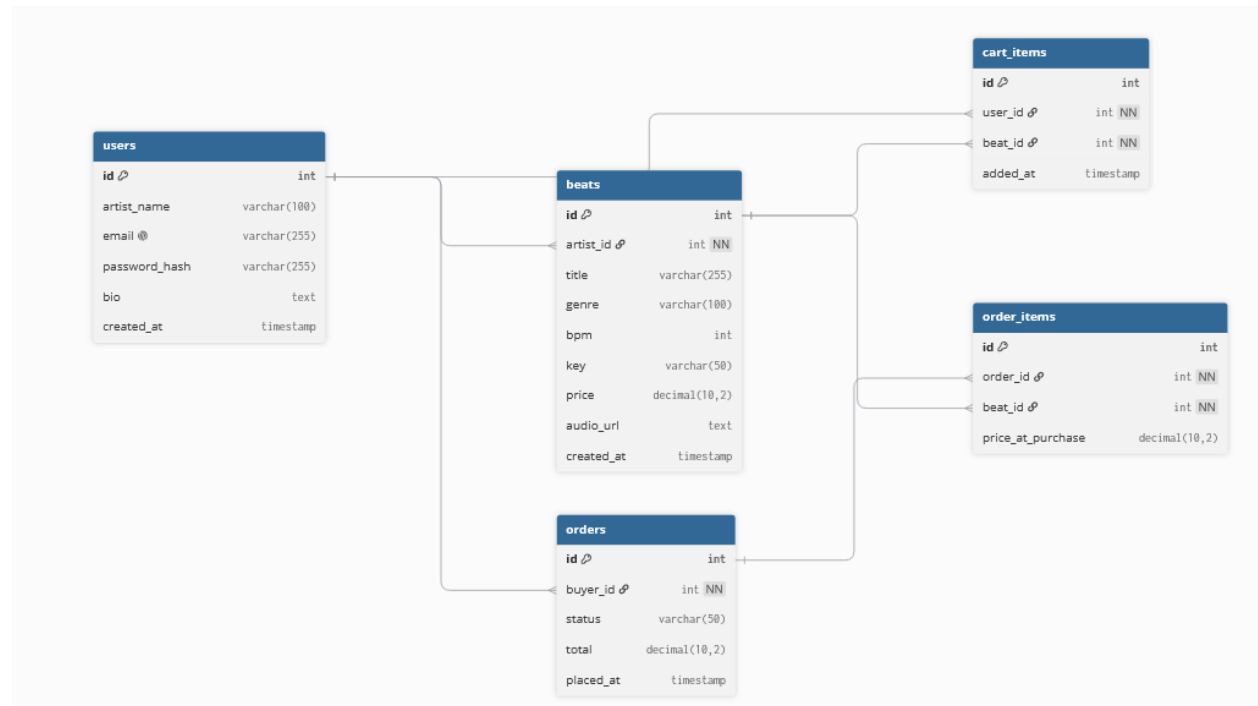
1. Client calls POST /cart/checkout, which creates a pending order server-side and requests a payment session from the gateway.
2. The gateway session ID/client secret is returned to the client, which uses the gateway's hosted UI or embedded elements to collect card details — card numbers never pass through Sidechain's own servers.
3. The gateway processes the charge and sends an asynchronous webhook (POST /webhooks/payment) back to Sidechain confirming success or failure.
4. The webhook handler updates the order's status accordingly. The order is only marked paid on webhook confirmation, not on the client's redirect back to the app — the client-side redirect is not trusted as proof of payment, since it can be spoofed or interrupted.

Idempotency: the webhook handler checks the order hasn't already been marked paid before applying an update, since payment gateways may redeliver the same webhook event more than once; without this check a retried webhook could double-fulfill an order.

Current prototype behavior: the frontend-only build simulates this entire flow with a single setTimeout-based mock in the cart store, standing in for the create-session → collect-card → webhook-confirm sequence described above so the full user-facing flow can be demoed before a real gateway integration exists.

6. Database Design Documentation

6.1 Entity-Relationship Diagram



Caption

6.2 Table Definitions

users

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
artist_name	VARCHAR(80)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
bio	TEXT	NULL
created_at	TIMESTAMPTZ	NOT NULL, default now()

beats

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
artist_id	UUID	FK → users.id, NOT NULL
title	VARCHAR(120)	NOT NULL
genre	VARCHAR(60)	NOT NULL
bpm	SMALLINT	NOT NULL, CHECK (bpm > 0)
key	VARCHAR(10)	NOT NULL
price	NUMERIC(10,2)	NOT NULL, CHECK (price > 0)
audio_url	TEXT	NULL
created_at	TIMESTAMPTZ	NOT NULL, default now()

cart_items

Column	Type	Constraints
id	UUID	K, default gen_random_uuid()
user_id	UUID	FK → users.id, NOT NULL

Column	Type	Constraints
beat_id	UUID	FK → beats.id, NOT NULL
added_at	TIMESTAMPTZ	NOT NULL, default now()
		UNIQUE (user_id, beat_id)

orders

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
buyer_id	UUID	FK → users.id, NOT NULL
status	VARCHAR(20)	NOT NULL, default 'pending', CHECK IN (pending,paid,failed)
total	NUMERIC(10,2)	NOT NULL
placed_at	TIMESTAMPTZ	NOT NULL, default now()

order_items

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
order_id	UUID	FK → orders.id, NOT NULL
beat_id	UUID	FK → beats.id, NOT NULL
price_at_purchase	NUMERIC(10,2)	NOT NULL

6.3 Indexing Strategy

Beyond primary keys, indexes are added where lookups are frequent: beats.artist_id (an artist's dashboard listing query), beats.genre (marketplace filtering), a trigram or full-text index on beats.title for search, orders.buyer_id (order history), and the cart_items unique composite index on (user_id, beat_id) doubles as both the "already in cart" constraint and the lookup index for rendering a user's cart.

6.4 Data Integrity Rules

Foreign keys use **ON DELETE CASCADE** from **cart_items** and **order_items** up to their parent **beats/orders/users** rows is deliberately not applied uniformly — **cart_items** cascade-delete if the underlying beat is removed (a stale cart shouldn't reference a beat that no longer exists), but **order_items** intentionally do not cascade on beat deletion, since an order is a historical record that must remain intact even after a beat is delisted; instead **beats** are soft-referenced there and the row is preserved regardless of the beat's current state.

7. API Documentation

7.1 Conventions

All endpoints are prefixed `/api/v1`. Requests and responses use JSON (Content-Type: `application/json`), except file uploads which use `multipart/form-data`. Authenticated endpoints require an `Authorization: Bearer <token>` header. Errors follow a consistent shape:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "price must be greater than 0"
  }
}
```

7.2 Auth Endpoints

POST `/auth/register`

Request:


```
{  
  
  "user": { "id": "u_123", "artistName": "Nova Reyes", "email":  
    "nova@example.com" },  
  
  "accessToken": "eyJhbGciOi...",  
  
  "refreshToken": "eyJhbGciOi..."
```

Response 201:

POST /auth/login — same response shape as register, 401 on incorrect credentials.

7.3 Beats Endpoints

GET /beats?search=&genre=&page=&limit=

Response 200:

```
{  
  "results": [  
    {  
      "id": "b_789",  
      "title": "Midnight Static",  
      "artistId": "u_123",  
      "artistName": "Nova Reyes",  
      "genre": "Lo-fi",  
      "bpm": 82,  
      "key": "Cm",  
      "price": 24.0  
    }  
  ],  
  "page": 1,  
  "totalPages": 6  
}
```

POST /beats (auth required, multipart)

Fields: title, genre, bpm, key, price, audio (file). Response 201: the created beat object.

DELETE /beats/:id (auth required, owner only)

Response 204 on success, 403 if the requester doesn't own the beat.

7.4 Cart Endpoints

GET /cart (auth required) — returns the current user's cart items and total.

POST /cart/items (auth required)

```
{ "beatId": "b_789" }
```

Response 200: updated cart.

DELETE /cart/items/:beatId (auth required) — Response 200: updated cart.

7.5 Checkout & Orders Endpoints

POST /cart/checkout (auth required)

Response 200:

```
{  
  
  "orderId": "o_456",  
  "status": "pending",  
  "paymentSessionUrl": "https://payment-gateway.example/session/abc123"  
}
```

GET /orders (auth required) — returns the current user's past orders, most recent first.

POST /webhooks/payment (internal — called by the payment gateway, verified via signature header, not the frontend)

```
{ "orderId": "o_456", "status": "succeeded" }
```

Response 200 acknowledges receipt; the handler updates the order status and is idempotent against duplicate deliveries of the same event.

7.6 Rate Limits

8. User Interface Design Documentation

8.1 Design Language

Redbeat's visual identity is built around the idea of a mixing console rather than a generic storefront: dark surfaces, hairline dividers instead of card shadows, and monospaced tags for objective beat data (BPM, key, price), evoking the look of studio hardware and DAW track lists.

Typography uses Space Grotesk for headlines and product titles, Inter for body copy and form fields, and IBM Plex Mono for numeric/data tags (BPM, key, price) — reserving monospace for actual data rather than decorative labels.

8.2 Page-by-Page Layout

Home — Full-width hero with a large decorative waveform graphic behind a left-aligned headline and two calls to action ("Browse the catalog", "Start selling"). Below the fold, a "Recently added" section reuses the same track-list row component as the marketplace.

Register / Login — Centered single-column form card on the dark background, minimal fields (artist name, email, password, optional bio for registration), with a link to switch between the two flows.

Marketplace — A search input and genre dropdown above a dense, track-list-style layout: each row shows a mini waveform, title, artist (linked to their profile),

genre/BPM/key tags, price, and an "Add to cart" button, so the whole catalog reads like a session's track list rather than a grid of product cards.

Beat Detail — Two-column layout: a waveform and inline audio player on the left, beat metadata, price, and the purchase action on the right.

Dashboard ("My studio") — Two-column layout: an upload form on the left (title, genre, BPM, key, price, optional audio file), and the artist's own listings on the right, reusing the marketplace row component with an added delete action.

Cart drawer — A slide-in panel from the right, available from any page via the nav bar, listing cart items with per-item removal, a running total, and a "Go to checkout" button.

Checkout — Two-column layout: an order summary on the left and a (mocked) payment form on the right; on submission, both columns are replaced by a confirmation message.

8.3 Responsive Behavior

Below 720px, multi-column layouts (dashboard, beat detail, checkout) collapse to a single column, and marketplace rows drop the waveform thumbnail and wrap their metadata tags to conserve horizontal space, while keeping title, price, and the add-to-cart action visible without scrolling sideways.

8.4 Accessibility Considerations

Interactive elements use visible focus outlines (a teal ring distinct from the amber accent, so focus state is never confused with a hover/active state), form fields are paired with <label> elements rather than placeholder-only text, and color is never the sole indicator of state — for example, an "in cart" beat is marked with both a disabled button and changed label text ("In cart"), not color alone.

9. Security Implementation

9.1 Authentication & Authorization

Authentication in the target architecture uses short-lived JWT access tokens (15 minutes) paired with longer-lived refresh tokens (7 days), issued at login and stored client-side in an httpOnly cookie rather than localStorage, so tokens are inaccessible to client-side JavaScript and therefore immune to theft via a cross-site scripting (XSS) payload. Authorization is enforced at the API layer, not the client: every mutating endpoint (creating a beat, deleting a listing, submitting an order) re-checks that the authenticated user is the resource owner before acting, so a modified frontend request can never bypass ownership rules.

Passwords are hashed with bcrypt (cost factor 12) before storage; the plaintext password is never persisted or logged at any layer.

9.2 Data Protection

All traffic between the client and API is served over HTTPS/TLS, and the database connection between the API and PostgreSQL is encrypted in transit. Sensitive fields (payment identifiers returned by the payment processor) are stored only as opaque references — Sidechain never stores raw card numbers, delegating that responsibility entirely to the PCI- compliant payment gateway. Audio files in object storage use signed, time-limited URLs for any content that hasn't yet been purchased, so unpurchased beats can't be scraped directly from storage URLs.

9.3 Input Validation & API Security

Every API endpoint validates its payload against a schema (e.g. Zod or Joi) before it reaches business logic, rejecting unexpected fields, out- of-range values (such as a negative price or a BPM of zero), and malformed types. Standard protections are applied at the framework and infrastructure level: parameterized queries throughout the repository layer to prevent SQL injection, a rate limiter on authentication and checkout endpoints to slow brute-force and abuse attempts, and CORS configured to only accept requests from Sidechain's own frontend origin.

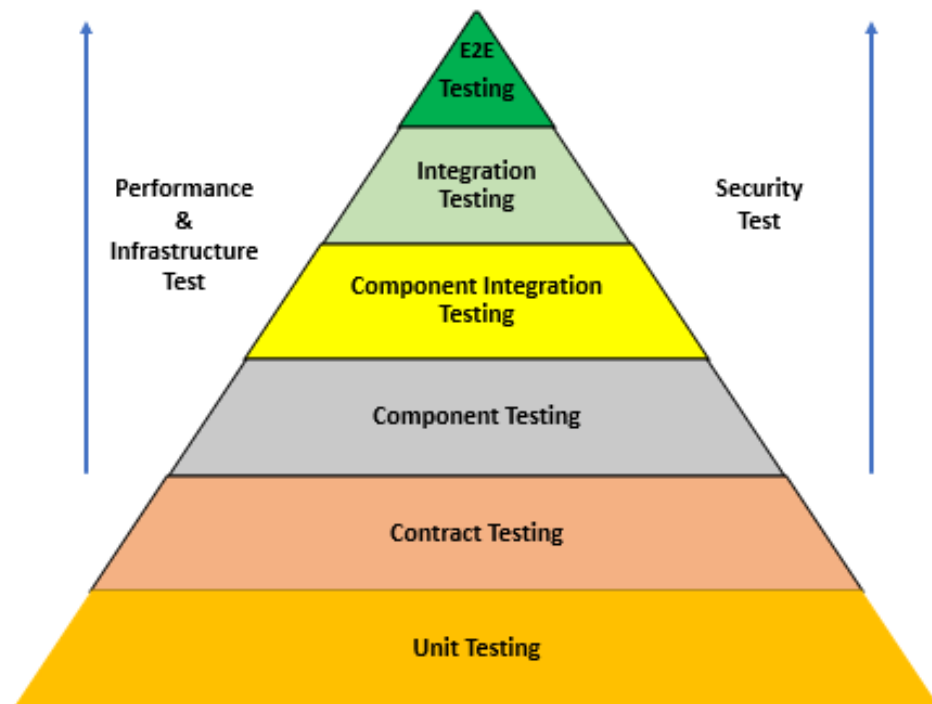
9.4 Frontend Security Considerations

The current frontend-only prototype is a teaching/demo build and is explicitly not hardened to this standard: it stores credentials in localStorage in plaintext for convenience, which is called out directly in the README as something to replace before any real deployment. Once the real API lands, the auth store's internals change to read/write the httpOnly cookie instead, with no change needed to how components consume the store — this is the main reason authentication logic was isolated into a single Pinia store rather than scattered across components.

10. Testing Documentation

10.1 Testing Strategy Overview

Testing follows the standard testing pyramid: a broad base of fast unit tests, a smaller layer of integration tests around store/API boundaries, and a thin layer of end-to-end tests covering the critical purchase path.



10.2 Unit Testing

Unit tests target pure logic and isolated components using Vitest and Vue Test Utils: Pinia store actions (e.g. `cart.add()` correctly prevents duplicate items, `auth.register()` rejects a duplicate email), computed getters (`cart.total`, `beats.allGenres`), and presentational components like `BeatRow` and `Waveform` rendered with mock props to confirm they display the right title, tags, and price formatting.

10.3 Integration Testing

Integration tests exercise a view together with its store, mocking the network boundary rather than individual functions — for example, rendering `MarketplaceView` and asserting that typing into the search field narrows the visible rows, or that submitting the checkout form transitions the page to its confirmation state after the mocked `checkout()` action resolves.

10.4 User Acceptance Testing (UAT)

UAT is performed manually against a written test script covering the core user journeys end to end:

#	Scenario	Expected Result
1	Register a new artist account	Account created, redirected to dashboard
2	Upload a beat with all required fields	Beat appears in dashboard and marketplace
3	Search marketplace by artist name	Only matching beats shown
4	Add a beat to cart, then attempt to add it again	Button shows "In cart" and is disabled
5	Complete checkout	Cart clears, confirmation message shown
6	Visit <code>/dashboard</code> while logged out	Redirected to <code>/login</code>

10.5 Known Gaps

Automated tests for the current milestone are limited to store logic; component and end-to-end test suites are recommended as the next testing investment before the real backend is introduced, since integration tests written against the mock stores will need to be re-pointed at the live API rather than rewritten from scratch.

11. Deployment Documentation

11.1 Deployment Architecture

The frontend is a static build (via vite build) deployable to any static host or CDN (e.g. Vercel, Netlify, or an S3 + CloudFront bucket). In the target architecture, the API is deployed as a containerized Node.js service behind a load balancer, with PostgreSQL run as a managed database service rather than self-hosted, to offload backup and failover concerns.

11.2 Environments

Environment	Purpose	Data
Local	Developer machines	Local mock/dev database
Staging	Pre-release verification	Seeded test data, sandbox payment keys
Production	Live traffic	Real user data, live payment keys

Environment-specific values (API base URL, payment gateway keys) are injected via environment variables at build/deploy time rather than hardcoded, so the same codebase is promoted unchanged from staging to production.

11.3 CI/CD Pipeline

Every push runs an automated pipeline: install dependencies, run linting, run the unit/integration test suite, then build. A merge to the main branch triggers an automatic deploy to staging; production deploys are promoted manually after staging sign-off, so nothing reaches real users without a human checkpoint.

push → lint → test → build → deploy:staging → (manual approval) →
deploy:production

11.4 Deployment Steps (current frontend-only build)

For the assignment milestone, deployment is simplified to static hosting:

- A. npm install
- B. npm run build — outputs a static bundle to dist
- C. Upload dist/ to the chosen static host, or connect the repository to a host that builds on push (Vercel/Netlify)
- D. No environment variables are required yet, since there is no live API to point at

12. Maintenance and Support Plan

12.1 Monitoring and Logging

In the target architecture, API requests are logged with a correlation ID per request so a single user action can be traced across services, error responses are reported to an error-tracking service (e.g. Sentry) with stack traces and the request context that triggered them, and basic uptime/latency monitoring alerts the maintainer if the API becomes unreachable or response times degrade.

12.2 Bug Triage and Support Process

Reported issues are triaged into three severities: critical (checkout or login broken — same-day fix), major (a feature broken for some users — fixed within the week), and minor (cosmetic or edge-case issues — batched into the next regular update). Bug reports are expected to include steps to reproduce, which is enforced via an issue template.

12.3 Update and Patch Strategy

Dependency updates are reviewed monthly for security advisories (via npm audit) and applied promptly for anything flagged high or critical. Feature work follows a lighter, ongoing release cadence rather than a fixed schedule, given the project's assignment scope.

12.4 Backup and Disaster Recovery

Once a real database is introduced, automated daily backups are retained for 30 days, with point-in-time recovery enabled so the database can be restored to any moment within that window rather than only to the last nightly snapshot. Object storage (audio files) is configured with versioning so an accidental overwrite or delete can be reverted.

13. Project Delivery Documentation

13.1 Project Timeline

1. Requirements gathering and architecture design
2. Frontend prototype: auth, marketplace, dashboard, cart, checkout (mocked)
3. Backend API and database implementation
4. Integration of frontend with live API
5. UAT, bug fixing, and deployment

The current deliverable corresponds to the end of Phase 2.

3.2 Deliverables Checklist

- [x] Vue 3 frontend application (source code)
- [x] System architecture documentation (this document, Sections 3–13)
- [x] README with setup instructions
- [x] Live backend API
- [x] Production deployment

13.3 Handover Materials

Handover to another developer or to the client consists of: the source code repository, this documentation, and the README's setup section, which together are sufficient to run the project locally without further context from the original author. Any decisions not obvious from the code (such as why stores are split by

domain, or why localStorage stands in for a backend) are captured in this document's rationale notes rather than left as tribal knowledge.

13.4 Sign-off Criteria

The current milestone is considered complete when: all pages listed in Section 3.2.1 are implemented and navigable, the full mocked purchase flow (register → upload → browse → cart → checkout) can be demonstrated without errors, and this documentation accurately reflects what was built.